

Bee_Fi

Multi-Agent Reinforcement Learning for Orbital Satellite Task Scheduling

Comprehensive System Guide

With Basilisk Orbital Dynamics Integration

Generated: March 06, 2026

Version: 2.0 (BSK Integration)

Platform: PettingZoo + PyTorch + Basilisk

Table of Contents

1.	System Overview	3
2.	Architecture	4
3.	Neural Network Design	6
4.	Environment (BeeForagingEnv)	8
5.	Basilisk Integration	12
6.	Task System & Metadata	15
7.	Gossip Communication Protocol	17
8.	Training Pipeline	18
9.	BSK Evaluator & Telemetry	21
10.	Configuration Reference	23
11.	Performance Results	24
12.	File Reference	25
13.	CLI Commands Reference	26
14.	Known Issues & Future Work	27

1. System Overview

Bee_Fi is a Multi-Agent Reinforcement Learning (MARL) system that simulates orbital satellite swarms performing coordinated task harvesting. The system uses a bio-inspired metaphor where satellites are modeled as 'bees' and observation tasks are modeled as 'flowers' with pollen values representing task priority.

The project combines orbital mechanics (Keplerian, SGP4, and Basilisk high-fidelity simulation), battery-driven fault tolerance, time-constrained task windows, decentralized gossip-based communication, and PPO-trained transformer attention policies for decision-making.

1.1 Key Capabilities

Orbital Task Scheduling: 25 satellites coordinate to complete 50 observation tasks on passive orbital trajectories

Battery Fau

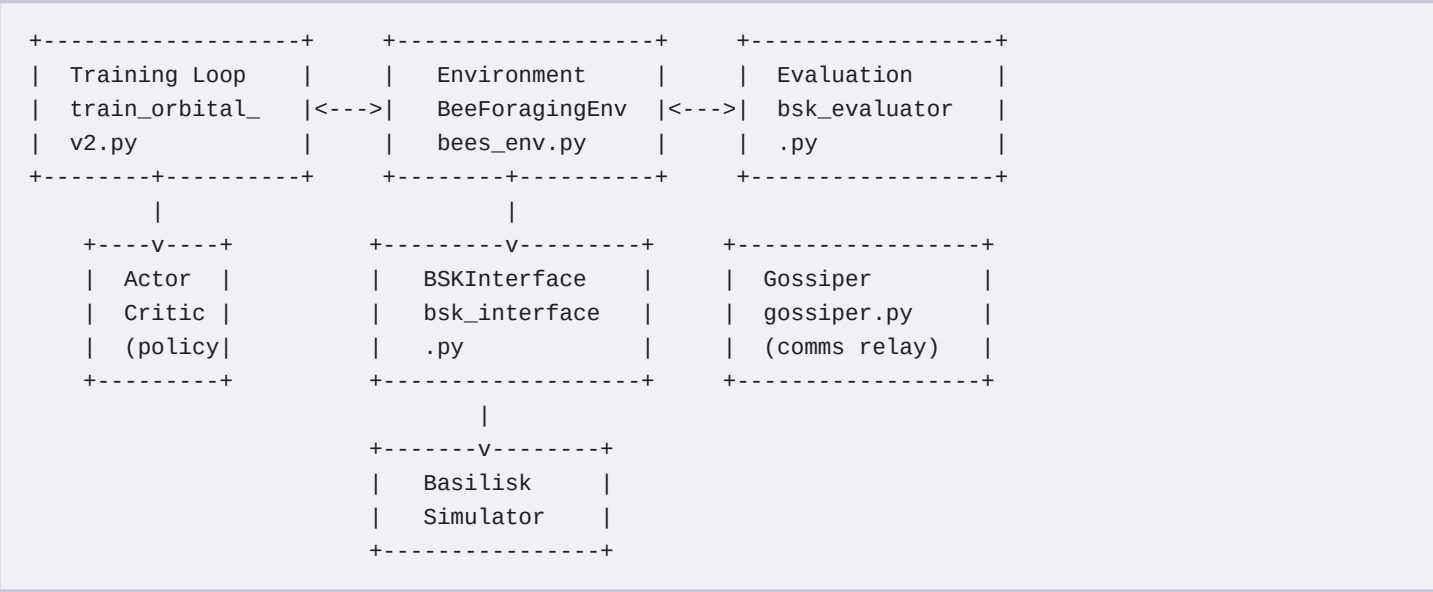
1.2 Technology Stack

Component	Technology	Version
RL Framework	PettingZoo (ParallelEnv)	>= 1.24.0
Deep Learning	PyTorch	>= 2.0.0
Orbital Dynamics	Basilisk (BSK)	Built from source
Orbital Propagation	SGP4 (optional)	>= 2.20
Training Algorithm	PPO (Proximal Policy Optimization)	Custom impl
Visualization	Matplotlib 3D + Dash	>= 3.5.0
Configuration	YAML	PyYAML >= 6.0
Logging	TensorBoard	>= 2.12.0

2. Architecture

The system follows a modular architecture with clear separation between the simulation environment, neural network policies, training loop, and evaluation pipeline.

2.1 High-Level Architecture



2.2 Data Flow

Each simulation step follows this pipeline:

Step 1: BSKInterface advances Basilisk by 1 second, returns ECI positions + battery for each satellite

Step 2: bee
me

2.3 Operating Modes

Mode	Dynamics	Use Case
Keplerian	Analytical 2-body orbits	Fast training (default)
SGP4	TLE-based propagation	Real satellite elements
Basilisk (BSK)	High-fidelity N-body + J2	Evaluation & BSK training

3. Neural Network Design

3.1 Actor Network (Per-Agent Policy)

The Actor is a transformer-attention-based policy network. Each bee receives its own local observation and independently selects an action. The architecture uses learned attention over flower features to focus on the most relevant tasks.

Feature Encoders (Input Stage)

Input	Raw Dim	Encoder	Output Dim
Position (x,y,z)	3	Linear + LayerNorm + ReLU	64
Status (load, capacity)	2	Linear + LayerNorm + ReLU	32
Flowers (N x 12 features)	12 per flower	Linear + LayerNorm + ReLU	128 per flower
Step count	1	Linear + LayerNorm + ReLU	32
Consensus (last actions)	N_bees	Linear + LayerNorm + ReLU	64
Retask board (M x 5)	5*M	Linear + LayerNorm + ReLU	128
Action availability	3	Linear + LayerNorm + ReLU	32

Flower Attention (Transformer)

Each flower's 12-feature vector is embedded to 128 dimensions, then processed by a 1-layer Transformer Encoder with 4 attention heads. The output is mean-pooled to produce a single 128-dimensional flower summary vector. This allows the agent to attend to the most relevant flowers (nearby, high-priority, approaching deadline).

Trunk Network

All encoded features are concatenated (total ~480 dims) and passed through two Linear layers: 480 -> 256 -> 256, each with LayerNorm and ReLU activation.

Output Heads

Head	Output	Description
Policy Head	3 logits	DONOTHING / HARVEST / GROOM probabilities
Claim Head	M+1 logits	Retask board slot selection (0..M-1) or no-claim

3.2 Centralized Critic Network

The critic observes the global state (all bees concatenated + all flower features) to estimate state value $V(s)$. This centralized training / decentralized execution (CTDE) paradigm gives the critic a full view while each actor only sees its local obs.

Architecture: Global state -> Linear($N \times \text{obs_dim}$, 512) + LayerNorm + ReLU -> Linear(512, 512) + LayerNorm + ReLU -> Linear(512, 1) -> $V(s)$ scalar.

Key Design Decision

At inference time, ONLY the Actor is used. The Critic is discarded.

This means deployed satellites only need the lightweight Actor network, not the large Critic that sees global state.

4. Environment (BeeForagingEnv)

BeeForagingEnv is a PettingZoo ParallelEnv subclass implementing the full multi-agent satellite task scheduling simulation. It is approximately 2,220 lines of Python code and handles orbital mechanics, battery management, task assignment, time windows, gossip communication, and reward computation.

4.1 Agents (Bees / Satellites)

Property	Default	Description
num_bees	25	Number of satellites in the constellation
Position	(fx, fy, fz)	3D grid coordinates (mapped from orbits or BSK)
Battery	450-750 steps	Randomized per episode; drains 1.0/step
Load / Capacity	0.0 / 10.0	Current pollen (task data) vs max capacity
Orbital Elements	a, e, i, Om, w	Keplerian elements (synced from BSK in BSK mode)
Mode	IDLE/HARVEST/GROO	Current activity state
Assigned Flowers	list[int]	Task queue assigned to this satellite

4.2 Tasks (Flowers)

Property	Type	Description
Position	(x, y)	Fixed on grid, placed along orbital paths
Pollen	float 0.8-10.0	Task value / priority weight
Priority	float 0-1	pollen / max_capacity
Time Window	HARD/SOFT/NONE	Scheduling constraint type
task_id	str	Unique ID: TASK-{id:03d}-{x}-{y}
task_description	str	One of 5 task types (imagery, relay, etc.)
status	str	unassigned / assigned / completed / expired
deadline_step	int	Absolute step deadline (priority-scaled)
created_step	int	Step when task was created

4.3 Action Space

Action	ID	Effect	Reward
DONOTHING	0	Continue orbiting	0 (small penalty if flowers nearby)
HARVEST	1	Collect pollen from nearest valid flower	+5.0 + pollen bonus (up to +10)
GROOM	2	Offload pollen or recharge battery	+5.0 to +15.0 (strategic)

4.4 Observation Space (Per Bee)

Each bee receives a dictionary observation with these components:

Field	Dimension	Source
position	3	(x,y,z) / grid_size
status	2	[load/capacity, capacity_norm]
battery	2	[battery_frac, is_recharging]
flowers	N_flowers x 12	12 features per flower (see below)
step_count	1	current_step / max_steps
consensus	N_bees	Last actions of all bees / 2.0
retask_board	M x 5	Orphan task features from gossip
action_availability	3	[can_harvest, can_groom, can_idle]

Per-Flower Features (12 each)

#	Feature	Description
0	x	Normalized grid x position
1	y	Normalized grid y position
2	pollen	Pollen amount / max capacity
3	harvested	1.0 if already harvested
4	mine	1.0 if assigned to this bee
5	busy	1.0 if another bee is currently harvesting
6	reachable	1.0 if on this bee's reachable orbital path
7	fits	1.0 if pollen fits in remaining capacity
8	dist_norm	3D distance / harvest_radius
9	harvestable_now	1.0 if within active time window
10	hard_window	1.0 if HARD time window type
11	time_to_window	Steps until next window / max_steps

4.5 Battery & Recharge System

Each bee has a randomized battery capacity (450-750 steps). Battery drains by 1.0 per step. When battery reaches 0, the bee 'dies' and enters recharge mode for 40 steps. During recharge, all assigned flowers are released to the unassigned pool and broadcast via gossip relay.

Curriculum learning: 5% of episodes use 'low-battery mode' with 1/3 capacity and 2-3x drain rate, forcing the policy to learn proactive recharge behavior.

4.6 Proactive Groom Mechanism

Before processing a HARVEST action, the environment checks if the bee's remaining capacity can fit the smallest reachable flower. If not, it automatically triggers a GROOM action instead, preventing wasted harvest attempts on full-capacity bees.

4.7 Task Assignment & Retasking

Initial Assignment: Round-robin by descending pollen value at episode start

Exclusivity:

4.8 Episode Termination

An episode ends when any of these conditions is met:

- All flowers are DONE (harvested or expired)

- All be

End states: SUCCESS (all harvested), PARTIAL (mix of harvested and expired), TIMEOUT (max steps).

5. Basilisk Integration

Basilisk (BSK) is a NASA-developed astrodynamics framework providing high-fidelity orbital simulation. Bee_Fi integrates BSK via the BSKInterface wrapper class, replacing the analytical Keplerian propagator with a full N-body + J2 perturbation model.

5.1 BSKInterface (bsk_interface.py)

Thin wrapper (~420 lines) around multi-satellite Basilisk simulation. Creates N spacecraft with:

Gravity Model: Earth central body with J2 perturbations

Battery: S

Default constellation: Walker-delta LEO at 550 km altitude, 53 degrees inclination, 3 orbital planes. This matches typical communication satellite constellations.

5.2 BSK Mode in BeeForagingEnv

Reset (Initialization)

When use_basilisk=True, the reset() method performs these additional steps:

- BSKInterface creates N spacecraft in Basilisk simulator

- get_p
meter

Step (Per-Tick BSK Updates)

Phase	What Happens
BSK Position Update	bsk.step() advances Basilisk by 1s; new ECI -> grid coords -> bee.fx/fy/fz
BSK Battery Sync	bsk_battery_frac * battery_max[i] overwrites env battery
Action Override	Battery-dead bees forced to DONOTHING
Proactive Groom	If HARVEST but can't fit smallest flower -> auto-groom
HARVEST Resolution	Find closest reachable, non-expired, assigned/unassigned flower
Task Expiration	Flowers past deadline_step -> expired, penalty to owner
Gossip Propagation	Dead/orphan tasks relay hop-by-hop

5.3 BSK vs RL Control Matrix

Aspect	BSK Simulator	RL Policy
Satellite Position	Yes - Full N-body mechanics	No - Overwritten by BSK

Velocity	Yes - Computed by BSK	No - In state but not in obs
Battery Drain	Yes - SimplePowerSink	No - Synced from BSK
Solar Recharge	Yes - SimpleSolarPanel	No - Automatic in BSK
Orbital Manoeuvring	No - No thruster model	No - Passive orbits
Task Selection	No	Yes - HARVEST/GROOM/DONOTHING
Task Assignment	No	Yes - Retask board + gossip
Groom Timing	No	Yes - When to offload/recharge
Task Expiration	No	Yes - Must harvest before deadline

Key Insight

The RL policy does NOT control orbital motion. Satellites follow their Basilisk-computed orbits passively. The policy's job is purely TASK SCHEDULING - given where each satellite IS (from BSK), choose whether to harvest, groom, or wait.

5.4 Element Sync Fix (0% to 54% Harvest)

Initial BSK integration yielded 0% harvest because flowers were placed using default Keplerian elements (wide, eccentric orbits) while BSK positioned satellites in a tight Walker constellation. The fix syncs Bee Kepler elements FROM BSK orbital elements after initialization, then re-replaces flowers along the corrected orbits. Result: harvest rate jumped from 0% to 54% with the Keplerian-trained model.

6. Task System & Metadata

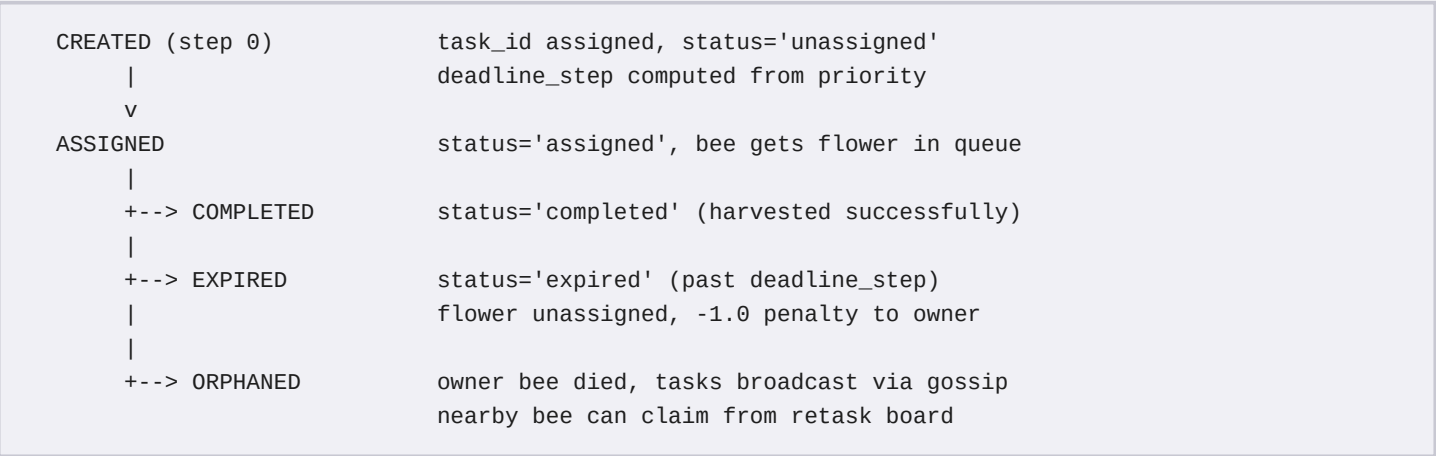
The task system was inspired by the BSU Autonomous Technologies Lab's `satellite_constellation_scheduling` repository. Each flower (task) carries rich metadata for realistic satellite task scheduling.

6.1 Task Descriptions

Tasks cycle through 5 description types based on flower index:

Index	Task Description	Analogy
0	Capture imagery	Earth observation / remote sensing
1	Relay communication	Inter-satellite link relay
2	Data collection	Sensor data aggregation
3	Sensor calibration	Instrument maintenance
4	Harvest pollen	General task completion

6.2 Task Lifecycle



6.3 Deadline Computation

Deadlines are priority-scaled: higher priority tasks have tighter deadlines. The formula is:

```
deadline_step = int(max_steps * (0.3 + 0.5 * (1 - priority)))
```

For `max_steps=1200`: a `priority=1.0` task expires at step 360, while a `priority=0.0` task expires at step 960. This forces the agent to prioritize high-value tasks.

6.4 Expiration Sweep

Every step, the environment sweeps all flowers. Any unharvested flower whose current step exceeds its `deadline_step` is marked expired. The owning bee receives a -1.0 penalty, and the flower is excluded from all future harvest scans, retask board entries, and action availability calculations.

7. Gossip Communication Protocol

Gossiper (gossiper.py, ~430 lines) implements a physics-based gossip relay protocol for decentralized task redistribution. When a satellite fails (battery=0), its orphan tasks are broadcast as GossipMessage objects that propagate hop-by-hop through the constellation.

7.1 GossipMessage Structure

Field	Type	Description
flower_id	int	Index of the orphaned flower/task
priority	float	Task priority (pollen / capacity)
origin_bee	int	Bee that died and released the task
sender_bee	int	Last bee that forwarded the message
x, y	float	Flower grid coordinates
pollen	float	Pollen value of the task
hops	int	Number of relay hops so far
ttl	int	Time-to-live (max hops before discard)
created_step	int	Step when message was created
seen_by	set[int]	Set of bee IDs that have seen this message

7.2 Propagation Rules

Range Limit: Messages only reach bees within $\text{harvest_radius} * 1.5$ distance

One Hop Pe

7.3 Retask Board Features (Per Slot)

Feature	Description
x	Normalized orphan flower x position
y	Normalized orphan flower y position
pollen	Pollen / max capacity
can_fit	1.0 if fits in bee's remaining capacity
hops	Gossip hop count / 10

8. Training Pipeline

8.1 PPO Algorithm

Proximal Policy Optimization (PPO) is used for stable multi-agent training. Key advantages: trust-region updates prevent catastrophic policy changes, clipped surrogate objective ensures monotonic improvement, and entropy bonus maintains exploration.

Training Loop Pseudocode

```
For update = 1 to 5000:
  1. Collect rollout (768 steps in environment)
    - For each step: observe, sample action, step env, store transition
  2. Compute GAE advantages:
    - TD error:  $d_t = r_t + \gamma * V(s_{t+1}) - V(s_t)$ 
    - Advantage:  $A_t = \sum(\gamma * \lambda^k * d_{t+k})$ 
    - Returns:  $G_t = A_t + V(s_t)$ 
  3. PPO update (2 epochs, minibatch_size=128):
    - Actor loss =  $-\min(\text{ratio} * A, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) * A) + \text{entropy}$ 
    - Critic loss =  $\text{MSE}(V(s), G)$ 
    - Backprop + Adam optimizer
  4. Checkpoint if best mean reward
  5. Early stop if harvest_rate >= 95% for 50 consecutive episodes
```

8.2 Hyperparameters

Parameter	Value	Description
num_updates	5000	Maximum training iterations
rollout_len	768	Steps per rollout before update
gamma	0.99	Discount factor
lambda (GAE)	0.95	Bias-variance tradeoff
clip_ratio	0.2	PPO clipping (+/- 20%)
value_coef	0.5	Critic loss weight
entropy_coef	0.02	Exploration bonus strength
lr_actor	0.0003	Actor learning rate (Adam)
lr_critic	0.0003	Critic learning rate (Adam)
num_epochs	2	PPO epochs per update
minibatch_size	128	Mini-batch size
patience	1000	Early stopping patience
min_harvest_rate	0.95	Target harvest rate for early stop

8.3 Curriculum Learning

5% of episodes use 'low-battery mode' with reduced capacity and increased drain. This forces the policy to learn

proactive recharge, quick decision-making, and graceful degradation when teammates fail. Without this curriculum, agents learn lazy strategies that fail under stress.

8.4 Training Modes

Mode	Command	Output Dir
Standard (Keplerian)	python train_orbital_v2.py --config config.yaml	outputs/
BSK Mode	python train_orbital_v2.py --config config.yaml --bsk	outputs_bsk/
HRL Mode	python train_orbital_v2.py --config config.yaml --hrl	outputs_hrl/
HRL + BSK	python train_orbital_v2.py --config config.yaml --hrl --bsk	outputs_hrl_bsk/

8.5 Model Checkpointing

Models are saved at multiple points during training:

- best_actor.pt / best_critic.pt - Best mean reward achieved so far
- upd{N} - Update {N} model

8.6 Resume Training

Training can be resumed from any checkpoint:

```
python train_orbital_v2.py --config config.yaml \
  --resume outputs/upd400 --start-update 401
```

9. BSK Evaluator & Telemetry

bsk_evaluator.py (~713 lines) runs the trained policy in BSK mode and produces telemetrybridge.json-format output for analysis and presentation.

9.1 Running the Evaluator

```
# Set Basilisk path
export PYTHONPATH="basilisk/dist3:$PYTHONPATH"

# Run BSK evaluation
python bsk_evaluator.py \
  --model outputs/best_actor.pt \
  --num_sats 25 --num_tasks 50 \
  --steps 1200 --snapshot_interval 60 \
  --output bsk_telemetry_eval.json

# Run without BSK (Keplerian fallback)
python bsk_evaluator.py \
  --model outputs/best_actor.pt \
  --num_sats 25 --num_tasks 50 \
  --no_basilisk \
  --output keplerian_eval.json
```

9.2 Telemetry Output Schema

Each snapshot includes per-satellite data:

Field	Type	Description
satellite_id	str	SAT-{id} identifier
status	str	active / dead / charging
orbit_type	str	NEO / MEO / GEO (based on altitude)
position_eci	dict	x, y, z in metres (from BSK)
velocity_eci	dict	vx, vy, vz in m/s
position_rn	dict	range, latitude, longitude
velocity_rn	dict	speed, heading
assigned_tasks	list	Each with task_id, description, priority, status
battery_level	float	0-100% charge
fuel_mass	float	Fuel in kg (static 50 kg, no thruster)
solar_panel_power	float	Solar output in watts
communication_status	str	online / offline
simulation_time	float	Basilisk sim time in seconds

9.3 Metrics Computed

Metric	Description
harvest_rate	Flowers harvested / total flowers
expired_count	Number of flowers that expired past deadline
alive_count	Satellites with battery > 0 at episode end
mean_battery	Average battery percentage across all satellites
gossip_messages	Total gossip messages sent during episode
episode_end_state	SUCCESS / PARTIAL / TIMEOUT
total_steps	Steps until episode termination

10. Configuration Reference (config.yaml)

10.1 Environment Settings

Parameter	Default	Description
num_bees	25	Number of satellites
num_flowers	50	Number of tasks
grid_size	75	World size (grid_size x grid_size)
max_steps	1200	Maximum episode length
harvest_radius	27.0	Euclidean harvest distance
reach_margin	4.5	Orbital reachability margin
lambda_z	0.5	Z-axis weight in 3D distance
orbit_scale	0.8	Orbital path scale factor
time_window_min	15	Earliest window start step
time_window_max	120	Latest window start step
battery_min_steps	450	Minimum battery capacity
battery_max_steps	750	Maximum battery capacity
recharge_steps	40	Steps to fully recharge
drain_per_step	1.0	Battery drain per step
low_battery_chance	0.05	Probability of low-battery episode
retask_board_size	3	Retask board slots per bee
retask_timeout_steps	40	Steps before bee is 'silent'
shaping_weight	0.05	Distance-based reward shaping weight
anti_spam_pen	-0.005	Invalid harvest attempt penalty
verbose	false	Detailed step logging

10.2 Training Settings

Parameter	Default	Description
hidden_dim	256	Network hidden layer size
num_updates	5000	Max training iterations
rollout_len	768	Steps per rollout
gamma	0.99	Discount factor
lam	0.95	GAE lambda
clip_ratio	0.2	PPO clipping ratio
value_coef	0.5	Value loss coefficient
entropy_coef	0.02	Entropy bonus coefficient

lr_actor	0.0003	Actor learning rate
lr_critic	0.0003	Critic learning rate
num_epochs	2	PPO epochs per update
minibatch_size	128	Mini-batch size
early_stopping	true	Enable early stopping
patience	1000	Early stopping patience
min_harvest_rate	0.95	Target harvest rate
convergence_window	50	Performance check window

11. Performance Results

11.1 Keplerian Mode (Training)

Trained with default Keplerian orbital dynamics, 25 bees, 50 flowers:

Metric	Value
Best Harvest Rate	93.9% (47/50 flowers)
Training Updates	~5000
Convergence	~500 updates to 80%, ~2000 to 90%+
Device	CUDA (GPU)
Training Time	~14 hours

11.2 BSK Evaluation (Keplerian-Trained Model)

The Keplerian-trained model was evaluated in BSK mode (Walker-delta constellation). Performance drops due to the geometry mismatch between training (spread Keplerian) and evaluation (tight Walker cluster).

Metric	Value
Harvest Rate	54% (27/50 flowers)
Expired Tasks	23
Alive Satellites	25/25 (100%)
Mean Battery	79.5%
Gossip Messages	0 (no deaths)
Episode End State	PARTIAL (step 1199)
Total Steps	1199

11.3 Performance Gap Analysis

The 54% harvest rate in BSK mode (vs 93.9% in Keplerian) is explained by:

- Tight Walker geometry: BSK satellites cluster in 3 orbital planes vs spread Keplerian orbits
- 'No va

Retraining directly on BSK mode (--bsk flag) is expected to close this gap significantly by learning Walker-specific scheduling behaviors.

11.4 BSK Training (In Progress)

A new model is currently being trained with use_basilisk=True to learn directly on BSK Walker constellation dynamics. This training uses the same hyperparameters but with Basilisk providing high-fidelity orbital positions and battery state.

12. File Reference

File	Lines	Purpose
bees_env.py	~2220	PettingZoo multi-agent environment (core simulation)
bee_state.py	~570	Bee + Flower classes with orbital mechanics + task metadata
bee_policy.py	~264	Actor (transformer attention) + CentralizedCritic networks
bsk_interface.py	~420	Basilisk wrapper - spacecraft creation, sim stepping
gossiper.py	~430	Physics-based gossip relay for task redistribution
train_orbital_v2.py	~1250	PPO training loop with BSK/HRL support
bsk_evaluator.py	~713	BSK evaluation + telemetrybridge.json output
telemetry_mapper.py	~400	Real telemetry JSON to simulation conversion
config.yaml	~90	All training + environment hyperparameters
bee_orbits_3d.py	~500+	3D matplotlib visualization with HUD
train_utils.py	~100+	Config loading, model save/load utilities
baseline_comparison.py	~200	Random, Greedy, Hungarian baselines
test_model_telemetry.py	~300	Evaluate trained model on real scenarios
mission_generator.py	-	Mission scenario generation
mission_runner.py	-	Mission execution runner

12.1 Output Directories

Directory	Contents
outputs/	Keplerian-trained model checkpoints (best_actor.pt, etc.)
outputs_bsk/	BSK-trained model checkpoints
outputs_hrl/	HRL-trained model checkpoints (manager + worker)
logs/	Training logs and HUD snapshots
basilisk/dist3/	Basilisk compiled distribution (built from source)

13. CLI Commands Reference

13.1 Training Commands

```
# Standard training (Keplerian mode)
python train_orbital_v2.py --config config.yaml --output outputs --seed 42

# BSK training (Basilisk orbital dynamics)
PYTHONPATH="basilisk/dist3:$PYTHONPATH" \
python train_orbital_v2.py --config config.yaml --output outputs --seed 42 --bsk

# HRL training (Hierarchical RL)
python train_orbital_v2.py --config config.yaml --output outputs --hrl

# Resume from checkpoint
python train_orbital_v2.py --config config.yaml --resume outputs/upd400 --start-update 401

# Force CPU
python train_orbital_v2.py --config config.yaml --cpu
```

13.2 Evaluation Commands

```
# BSK evaluation with trained model
PYTHONPATH="basilisk/dist3:$PYTHONPATH" \
python bsk_evaluator.py \
  --model outputs/best_actor.pt \
  --num_sats 25 --num_tasks 50 \
  --steps 1200 --snapshot_interval 60 \
  --output bsk_telemetry_eval.json

# Keplerian evaluation (no BSK)
PYTHONPATH="basilisk/dist3:$PYTHONPATH" \
python bsk_evaluator.py \
  --model outputs/best_actor.pt \
  --no_basilisk \
  --output keplerian_eval.json
```

13.3 Visualization Commands

```
# 3D orbital visualization with trained policy
python bee_orbits_3d.py --policy --model_tag best --stochastic

# TensorBoard for training curves
tensorboard --logdir outputs/
```

13.4 CLI Flags Reference

Flag	Type	Default	Description
--config	str	config.yaml	Path to YAML config file

--output	str	from config	Output directory override
--seed	int	42	Random seed for reproducibility
--cpu	flag	false	Force CPU execution
--bsk	flag	false	Enable Basilisk orbital dynamics
--hrl	flag	false	Use Hierarchical RL (Manager+Worker)
--resume	str	None	Checkpoint directory to resume from
--start-update	int	1	Starting update number when resuming
--manager-interval	int	10	Steps between manager decisions (HRL)

14. Known Issues & Future Work

14.1 Known Issues

'No Valid Target' Harvest Attempts

In BSK mode, the Keplerian-trained model frequently selects HARVEST when no flower is within harvest_radius. Root causes: (1) tight Walker geometry places fewer flowers within range, (2) expired flowers are invisible to the scan, (3) the model was trained on Keplerian orbits where flowers are more accessible.

Proposed solutions:

- Action masking: override HARVEST -> DONOTHING when no valid target exists

- Retra

Velocity/Fuel Not in Observations

BSK provides velocity and fuel mass per satellite, but these are not currently mapped into the observation space. Adding velocity could help the policy predict future positions and make better scheduling decisions.

Zero Gossip Messages in BSK Eval

In BSK evaluation, 0 gossip messages were sent because no bees died (79.5% mean battery). The death-triggered gossip pathway is not exercised in the current BSK configuration. Low-battery episodes would trigger gossip.

14.2 Future Work

BSK-Trained Model: Complete BSK mode training and evaluate (currently in progress)

Action Mas

Appendix A: Reward Structure Detail

Action / Event	Reward	Condition
Harvest success	+5.0 + pollen/5	Flower in range, not expired, assigned/unassigned
Pollen bonus	up to +10.0	Higher pollen value = higher bonus
Strategic groom	+5.0 to +15.0	Load >= 80% capacity, good timing
Strategic recharge	+8.0 to +13.5	Battery < 20%, proactive recharge
Proactive groom	+reward	Auto-groom when capacity can't fit flower
Unnecessary groom	-0.3	Groom when load < 10%
Invalid harvest	-0.05 to -0.2	No valid target within range
Miss HARD window	-50.0	HARD window closed permanently
Task expiration	-1.0	Flower passed deadline_step
Time decay	-0.01/step	Encourages faster task completion
DONOTHING w/ nearby flower	small penalty	Incentivizes action

Appendix B: Dependencies

```
# requirements.txt
torch>=2.0.0
numpy>=1.21.0
scipy>=1.9.0
sgp4>=2.20
matplotlib>=3.5.0
Pillow>=9.0.0
dash>=2.14.0
dash-bootstrap-components>=1.5.0
plotly>=5.18.0
pettingzoo>=1.24.0
gymnasium>=0.28.0
tqdm>=4.64.0
tensorboard>=2.12.0
pyyaml>=6.0
```

Appendix C: Basilisk Setup

Basilisk is built from source and installed at basilisk/dist3/:

```
# Build Basilisk from source (one-time)
cd basilisk
python conanfile.py
# ... (follow BSK build instructions)

# Set PYTHONPATH before running BSK-enabled commands
export PYTHONPATH="/path/to/Bee_Fi/basilisk/dist3:$PYTHONPATH"

# Verify BSK is available
python -c "from Basilisk.utilities import SimulationBaseClass; print('BSK OK')"
```